МГТУ им. Н.Э.Баумана

Кафедра ИУ7

«Проектирование программного обеспечения»

Сезон 2026

Лабораторная работа № 2

Проектирование

UML и C4

Подготовить в репозитории ветку *lab2* и создать merge request в ветку *master*.

Все пункты продолжать выполнять в Readme.md.

1. Описание типа приложения с точки зрения взаимодействия с пользователем и выбранного технологического стека (Console App/Desktop/Mobile/Web MPA/ Web SPA/Web PWA/Cross-platform/ Messenger(social) app и т.д.);

2. Подготовить раскрытие проекта на уровнях L1 и L2 нотации C4. Обратите внимание, что на уровне L2 даже на примере десктопного приложения у вас как минимум выделяется два контейнера: приложение и СУБД, а в случае web, - три (backend, frontend и СУБД).

3. Подготовить раскрытие проекта на уровне L3 нотации C4. Этот уровень в контексте настоящей лабораторной работы предполагает верхнеуровневое разбиение контейнеров на компоненты (в следующих лабах сможете уточнить): на базовом уровне выделяем компоненты доступа к данным, бизнес-логики и компоненты реализации UI. Подумать про *принцип инверсии зависимостей*.

4. Подготовить раскрытие проекта на уровне L4 нотации C4. Уровень кода в контексте настоящей ЛР раскрыть в виде UML диаграмм классов для как минимум двух компонентов - компонента доступа к данным и компонента с бизнес-логикой (не забыть «модельные» классов сущностей: сущности базы данных, сущности системы и транспортные сущности. Если уровень транспортных сущностей пока сложно спроектировать, можно скопировать системные. Уточните их на следующих этапах).

5. Подготовить диаграммы последовательностей в нотации UML для подготовленных в предыдущей ЛР BPMN-диаграмм. Для отрисовки диаграмм использовать PlantUML с сохранением исходников в репозитории.

6. Подготовить диаграмму БД с учетом выбранной СУБД в нотации DBML. Исходники сохранить в репозитории.

> **Примечание 1.** Для реализации компонента доступа к данным рекомендуется рассмотреть использование паттерна Repository и его многочисленных аналогов.
>
> **Примечание 2.** Для реализации компонента UI и его связи с остальным приложением рекомендуется рассмотреть использование MV*-подобного паттерна (MVC, MTV, MVP, MVVM, MVPVM, MVI и др.).
>
> **Примечание 3:** Если вы считаете, что бизнес логику логичнее разделить на несколько компонентов - разделяйте, но укажите причины этого решения.

Требования к технологическому стеку.

Рекомендуемые ЯП для использования в ЛР

Требование: для целей настоящего курса язык программирования должен быть полноценным ООП-языком со статической типизацией и поддержкой полиморфизма на базе интерфейсов (полностью абстрактных классов), наследования и инкапсуляции.

1. Рекомендуемые языки:
C#, Java, Kotlin, Scala, Swift, Objective C, C++, TypeScript, Dart, Delphi

2. Допустимые не полноценно ООП-языки (с учетом согласования с семинаристом способов реализации слоев луковичной чистой архитектуры, интерфейсов, инверсии зависимостей, компонентов и т.д.)
Go, Rust

3. Условно разрешенные языки (с учетом согласования с семинаристом)

Python
При этом, требуется неукоснительное следование рекомендациям: Методичка "Рекомендации по применению ЯП Python в курсе ППО" Жаров С.В.

4. Запрещенные языки
Ruby, PHP, Perl, JavaScript, 1C, Lua

~~5. Вне конкурса~~
~~*Haskell, Erlang, C*~~

Примечание:
Если планируется компонент UI на базе SPA-подхода, то вариант для этого компонента строго один - TypeScript, и компонентный фреймворк (Angular, React, Vue либо самописный). При этом остальная часть ПО может быть выполнена на другом языке.